

ShopSphere

S0 - Foundation and Engineering Baseline

Backend + DB

Platform-Specific Sprint Planning Document

This document intentionally defines only the behavior and delivery scope of the selected platform. It does not mix implementation details from the other platforms.

1. Sprint Definition

Sprint: S0 - Foundation and Engineering Baseline

Platform: Backend + DB

Application: ShopSphere

Objective: Create the backend and database foundation that every later business feature will depend on.

This sprint is complete only when the stated use cases, acceptance criteria and tests pass for this platform.

2. Business Use Cases

ID	Use Case	Actor	Precondition	Main Flow	Expected Result
UC-BE-00-01	Start service	Developer	Configuration is present	Start API, load configuration, connect to PostgreSQL/Redis, expose health endpoints	Service starts or fails fast with a controlled configuration error.
UC-BE-00-02	Run database migration	Developer	Database is reachable	Run migration, verify schema, record migration state	Migration completes without destructive implicit changes.

3. Detailed Implementation Plan

1. Create repository structure and module boundaries.
2. Configure NestJS bootstrap, configuration validation and dependency injection.
3. Configure PostgreSQL connection and Prisma schema/migrations.
4. Configure Redis client abstraction.
5. Create standard success/error response envelopes.
6. Add global exception handling, validation and correlation/request ID.
7. Add health/readiness endpoints.
8. Configure Swagger/OpenAPI.
9. Create Dockerfile and local docker-compose stack.
10. Create CI stages for lint, type-check, unit tests, build and migration validation.

4. Data / State / Contract Scope

- Initial Prisma migration metadata only; business tables are introduced by later feature sprints.
- Environment-specific configuration must be isolated from source code.

API / Integration Contract

- GET /health
- GET /ready
- OpenAPI document endpoint

5. Rules and Expected Behavior

- The platform must rely on the backend as the source of truth for business rules.
- Do not expose stack traces, secrets or internal implementation details to end users.
- Handle loading, empty, validation and error states explicitly.
- Requests that can be retried must be safe from duplicate side effects where the feature requires idempotency.
- Ownership and authorization must be checked server-side for protected data.

6. Acceptance Criteria

- API starts with valid configuration.

- Invalid configuration fails fast and clearly.
- PostgreSQL migration workflow works repeatedly.
- Redis connectivity is health-checked.
- Every API request receives a correlation/request identifier.
- CI blocks merges when required checks fail.

7. Test Cases

TC	Scenario	Expected Result
TC-S0-01	Missing database URL	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S0-02	Invalid Redis URL	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S0-03	Database unavailable	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S0-04	Redis unavailable	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S0-05	Malformed request	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S0-06	Unhandled exception returns controlled error	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S0-07	Migration rerun	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S0-08	CI lint failure	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S0-09	CI test failure	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.

8. Sprint Exit Checklist

- Implementation completed only for this platform.
- All primary and negative use cases pass.
- API contracts are implemented/consumed as defined.
- Validation and authorization tests pass.
- Loading/empty/error handling is verified where applicable.
- No critical or blocker defect remains.
- Automated tests pass in CI.
- Documentation and contract changes are committed.

9. Out of Scope for This Document

- Implementation behavior of Customer Portal. That behavior belongs in the separate S0 document for that platform.
- Implementation behavior of Admin Portal. That behavior belongs in the separate S0 document for that platform.